Carbon Commute Calculator

Project Overview

The `Carbon Commute Calculator` is a web-based application that estimates transportation-related carbon emissions based on a user's trip information and selected vehicle.

Users can:

- Add vehicles to a personal vehicle list
- Plan a commute using origin and destination addresses
- Visualize routes using Google Maps
- Calculate estimated CO_2 emissions
- Compare total emissions and emissions per passenger

This project promotes environmental awareness by helping users understand the environmental impact of daily transportation choices.

🦍 Authors

- Isabella Velazquez-Abelardo and Bhargavi Kale
- **Course:** COMP 705/805— Full-stack Development
- **Instructor:** Mihaela Sabin
- **Semester:** Spring 2026

🚀 Features

- Vehicle management system (SQLAlchemy)
- Google Maps route visualization
- Carbon emission calculations
- Passenger impact analysis
- Responsive multi-panel UI
- Flask backend with REST endpoints

🛠️ Technologies Used

- Python
- Flask
- SQLite
- SQLAlchemy
- HTML5 / CSS3
- JavaScript
- Google Maps API
- Pandas (vehicle dataset processing)

How Emissions Are Calculated

```
Total Emissions = Distance (km) × Emission Factor
Emissions per Person = Total Emissions ÷ Passengers
```

Emission Factors by Fuel Type

Fuel Type	kg CO_2 / km
Gasoline	0.20
Diesel	0.18
Hybrid	0.12
Electric	0.05

- Fuel type is automatically determined from the EPA vehicle dataset based on the user vehicle.

Project Structure

```
carbon-commute/
│
├── data/
│   └── vehicles.csv
├── docs/
│   └── sprints.md
│   └── Projectoutline.md
├── instance/
│   └── carbon_commute.db
├── static/
│   └── style.css
├── templates/
│   ├── home.html
│   └── index.html
│   └── vehicle.html
├── venv
├── .env
├── .gitignore
├── dev-requirements.txt
├── main.py
├── models.py
├── README.md
├── requirements.txt
```

🙏 Acknowledgments

- Original project inspiration and open-source resources referenced via GitHub repositories
- EPA vehicle emissions datasets
- Google Maps Platform API for mapping and routing services

Installation Guide

Google Cloud Setup

1. After creating your Google Cloud account, create your first Project on Google Cloud via the Google Cloud console (https://cloud.google.com/cloud-console)

2. Setup a Billing Account (credit card required)

- Free tier includes 10,000 API calls, more than enough for simple usage of this program.
- Please refer to the Google Maps Platform core services pricing list (https://developers.google.com/maps/billing-and-pricing/pricing)

3. Enable Routes API on your google cloud project

- https://console.cloud.google.com/apis/library/routes.googleapis.com?hl=en-GB

Requirements

- Python 3.10+
- pip
- Git
- Google Maps API Key

Step 1 — Clone Repository

- git clone https://github.com/2026-spring-comp-705-805/project-bella-bhargavi.git
- cd carbon-commute

Step 2 — Create Virtual Environment

- python -m venv venv
  Activate environment:

Windows
venv\Scripts\activate

Mac/Linux
source venv/bin/activate

Step 3 — Install Dependencies

pip install -r requirements.txt
pip install -r dev-requirements.txt

Step 4 — Environment Variables

Create `.env`
GOOGLE_API_KEY=your_api_key_here

Step 5 — Run Application

- python main.py
  Open browser:
  http://127.0.0.1:5000

Database

SQLite database is created automatically: `carbon_commute.db`

Test/lint/type-check commands

- Format Code: ruff format .
- Lint Code: ruff check .
- Type Checking: mypy .
- Static Analysis: pylint main.py models.py

System Architecture

Components

Frontend

- HTML templates (Jinja2)
- CSS responsive layout
- JavaScript Google Maps integration

Handles:

- User input
- Map rendering
- Displaying calculation results

Backend

Flask routes manage application logic:

Route	Purpose

Route	Purpose
`/`	Home interface
`/calculate`	Emissions calculation
`/add-vehicle`	Vehicle entry form
`/save-vehicle`	Store vehicle in database
`/api/vehicles`	JSON vehicle data

Database Layer

SQLite + SQLAlchemy ORM

Stores: Vehicle Information and Trip details and its co2 emission

Data Processing

Pandas reads EPA vehicle dataset:
`data/vehicles.csv`

Used to:

- Automatically determine fuel type
- Apply emission factors

Data Flow

- Vehicle Registration: User navigates to the "Add Vehicle" page to select a Make, Model, and Year.
- Database Storage: Backend fetches fuel type and MPG from the EPA dataset and saves them to the SQLite database.
- Trip Input: User selects a previously saved vehicle from the home page and enters trip details.
- Distance Calculation: The application uses the Google Routes API to determine driving distance between addresses.
- Emissions Logic: Backend calculates CO2 based on the saved vehicle's specific fuel type and trip distance.
- Visualization: The route is rendered on an interactive Google Map and results are shown in the UI panel.

Design Goals

- Simple architecture
- Clear separation of concerns
- Expandable for dashboards and analytics